

GAME DESIGN DOCUMENT

Sky Shooter: Save the City

Systems & Mechanics Design Sample

Prepared by: Nagesh Ranbhise

Game Designer • Unity Developer • 2D/3D Digital Artist

Portfolio Sample — demonstrating system documentation, combat design, progression thinking, economy structure, and implementation-ready mechanics.

1. Game Overview

Genre: 2D mobile arcade shooter with 3D assets **Platform:** Android **Engine:** Unity

High Concept

The player pilots a combat aircraft through increasingly dangerous environments, destroys enemy formations, avoids projectiles and environmental hazards, and progresses through themed levels. The design goal is immediate accessibility with enough tactical depth to reward skilled movement, targeting, and risk management.

Core Player Loop

1. Enter level → 2. Move and aim → 3. Destroy enemies → 4. Avoid damage → 5. Earn score/rewards → 6. Complete objective → 7. Unlock the next level/theme → 8. Replay to improve performance.

Design Pillars

- **Readable combat:** attacks and threats should be visually understandable before damage occurs.
- **Risk vs. reward:** aggressive positioning can improve score and efficiency, but mistakes remain punishable.
- **Progression:** players should feel increasingly capable without invalidating earlier challenge.
- **Responsive controls:** movement and targeting must feel predictable on mobile devices.

2. Combat System Specification

Player Movement

The aircraft uses joystick-based movement with configurable speed, rotation/tilt response, screen padding, and automatic forward motion. Movement should preserve player agency while preventing accidental movement outside the playable area.

Primary Attack

The primary weapon provides reliable continuous damage. Fire rate, projectile speed, damage, and enemy hit response are tunable values. The player should understand the relationship between positioning and damage output rather than relying only on raw weapon power.

Lock-On Missile

A lock-on system rewards deliberate target acquisition. Example specification: lock duration = 0.5 s; maximum lock distance = 20 units; lock angle = 35°. Once locked, the player can launch a missile toward the selected target. These values are balancing variables rather than hard-coded design assumptions.

Combat Feedback

Every successful hit should communicate impact through a combination of hit reaction, projectile impact, sound, score feedback, and/or enemy health change. Critical events such as player damage, lock confirmation, and enemy defeat receive stronger feedback.

3. Enemy & Boss Design

Enemy Archetypes

Archetype	Primary Role	Player Counter
Basic Shooter	Creates ranged pressure	Movement + prioritize target
Bomber	Area denial / burst threat	Maintain distance / destroy quickly
Formation Enemy	Controls space through patterns	Read formation and reposition
Elite	Tests sustained combat skill	Target priority + missile/weapon use

Boss Phase 1: Three-Move Pattern

1. Ground AoE Smash — Tell: Boss raises both arms; a circular warning indicator expands for approximately 1 second. **Counter:** move outside the marked radius.

2. Dash — Tell: Boss faces the player and briefly locks direction; a directional line indicates the dash path for approximately 0.7 seconds. **Counter:** dodge sideways.

3. Grab — Tell: Boss pauses and lowers its body; its hand glows and a short-range cone appears. **Counter:** roll or move laterally outside the cone.

Pacing Rule: Telegraphs must be consistent enough to learn, but attacks should be sequenced so the player cannot safely stand in one position throughout the encounter.

4. Risk vs. Reward Mechanic

Resolve Meter

Successful attacks and high-skill actions such as accurate targeting or parrying build Resolve. Taking damage reduces Resolve. A full meter can be spent for a small health recovery or temporary offensive benefit.

Balance Constraints

- Resolve decays when the player disengages, encouraging controlled aggression.
- Healing is capped and cannot replace normal health management.
- The activation window creates vulnerability, preventing free healing during dangerous attacks.
- The mechanic rewards skilled players without removing the consequences of reckless play.

5. Progression & Economy Model

XP Philosophy

XP rewards scale with enemy difficulty, while XP requirements also increase by level. The economy is designed around the player's expected progression rather than simply making every stronger enemy drop proportionally more XP.

Illustrative Economy Table

Content	Example XP	Purpose
Level 1 Skeleton / Basic Enemy	50	Early progression reference
Elite Enemy	500–1,500	Meaningful mid-stage reward
Level 25 Boss	10,000	Major milestone
Level 50 Boss	50,000	End-game milestone

These numbers are placeholders for balancing. The economy spreadsheet would calculate XP required per level, average XP/minute, expected level at each chapter, farming efficiency, and the effect of optional content.

Balancing Checks

- Define target player level for each major chapter.
- Simulate normal, highly skilled, and farming-focused playstyles.
- Check whether bosses create unintended multi-level jumps.
- Ensure new abilities feel powerful while enemies retain meaningful counterplay.
- Use telemetry after release to compare predicted progression with actual player behavior.

6. Time-Rewind Ability — Implementation Specification

Purpose: Give the player a controlled recovery tool that rewards planning without becoming a free escape from every mistake.

Primary Flow

INPUT → Check cooldown/resource → Start rewind → Restore recorded game state → End rewind → Lock ability → Start cooldown → Return to normal gameplay.

Proposed Values

Parameter	Initial Value
Recorded history	5 seconds
Cooldown	15 seconds
Activation	Player ability button
Affected state	Player position, health, selected enemy/projectile states
Restrictions	Cannot bypass checkpoints or permanent progression

Edge Cases to Define

- What happens if the player dies during the rewind window?

- Are newly spawned enemies removed or retained?
- Are boss projectiles restored to their previous positions?
- Can the ability be activated while stunned, during a cutscene, or at a checkpoint?
- Does rewinding restore consumed resources or only world/player state?

Acceptance Criteria

The mechanic is complete when the player can reliably rewind the defined state window, cooldown prevents repeated use, permanent progression is not duplicated or lost, and all documented edge cases produce deterministic results.

7. Designer–Programmer Handoff

A mechanic is considered implementation-ready when the designer provides measurable rules rather than subjective instructions such as “make it cool.” Each feature should contain:

- **Player flow:** inputs, states, transitions, and expected outcome.
- **Numbers:** duration, cooldown, range, damage, speed, costs, limits.
- **Visual/audio tells:** what the player sees or hears and when.
- **Edge cases:** unusual interactions that must be explicitly handled.
- **Acceptance criteria:** objective conditions that define “done.”
- **Balancing variables:** values exposed for iteration rather than buried in code.

8. Design Summary

My approach to game design combines player experience with implementation clarity. I use measurable rules, readable telegraphs, progression targets, economy simulations, edge-case documentation, and acceptance criteria so that designers and programmers share the same interpretation of a system.

This sample demonstrates how I would document a mechanic from concept through balancing and programmer handoff, while keeping the design flexible enough for iteration during development.

Nagesh Ranbhise

Game Designer • Unity Developer • 2D/3D Digital Artist